Afra B Project Delivery Documentation

Frontend Rebuild, Backend Integration, Checkout/Logistics Work, and PM Coordination

Prepared for: Afra B CEO / Payment Processing Review

Prepared by: David Efe Erigbe

Date: July 21, 2026

This document summarizes the full scope of work delivered across frontend, backend, product management, vendor coordination, QA, deployment readiness, and production hardening for the Afra B marketplace application.

Important compensation context: the engagement scope went beyond the usual PM and backend responsibilities. A complete production-ready frontend rebuild was also delivered, including API integration, checkout, logistics, payments, seller account tooling, messaging, wallet flows, and deployment preparation.

1. Executive Summary

The Afra B marketplace frontend was originally exported from Lovable and needed to be transformed into a stable local project that could run independently, integrate correctly with the Afra B core backend, support real user workflows, and be deployable to a public host such as Vercel.

The work delivered in this project was not limited to small frontend edits. It included production-critical marketplace flows: login/signup, product listing, product editing, multi-image upload, search suggestions, cart quantity and stock enforcement, checkout with Paystack, seller pickup and buyer delivery addresses, Kwik delivery quote and dispatch tracking support, messaging, wallet withdrawal, bank resolution, saved bank visibility, order history, purchase/sales tracking, and frontend deployment migration away from Lovable.

In parallel, backend behavior was clarified and implemented or coordinated around the updated checkout, fee, settlement, message, bank, withdrawal, and email verification flows. PM work also included repeated vendor coordination and follow-ups with Kwik to unblock logistics quoting, staging dispatch, and tracking behavior.

Workstream	Summary of Delivered Value
Frontend rebuild	Converted the exported app into a practical production frontend with integrated marketplace, seller, buyer, wallet, checkout, messaging, and deployment flows.
Backend/API integration	Aligned frontend behavior to backend Swagger/API contracts, especially checkout, fees, addresses, payments, messages, wallet, banks, and product listing.
Product management	Translated changing business/backend requirements into frontend behavior, identified blockers, clarified flows, and coordinated logistics/payment readiness.
Vendor coordination	Managed repeated follow-up and issue clarification with Kwik around staging logistics, quote/dispatch behavior, and tracking readiness.
Production readiness	Removed Lovable deployment dependency, added README/start instructions, ran build/lint checks, and prepared Vercel-compatible build output.

2. Engagement Context and Scope Expansion

The original professional value brought into this project was primarily PM and backend delivery. However, as the project evolved, the frontend became a major dependency for launch readiness. The exported Lovable app could not remain as an unfinished prototype because buyers, sellers, and admins needed real production workflows.

As a result, a full frontend rebuild and production integration effort was delivered in addition to backend and PM work. This included practical UX decisions, API wiring, state handling, validation, payment and delivery handling, seller tooling, account pages, deployment preparation, and repeated fixes based on real testing feedback.

This extra frontend delivery materially increased the scope of the engagement. It converted the project from a visually acceptable prototype into a frontend that can actually be used against the Afra B backend.

3. Frontend Work Delivered

3.1 Local Project Stabilization

- Exported Lovable frontend files were stabilized as a local project.
- Confirmed the app could run locally and documented startup instructions in the README.
- Connected the frontend to the Afra B backend base URL through environment configuration.
- Audited the app against the backend Swagger/API surface and progressively linked missing flows.

3.2 Production UI Cleanup

- Removed prototype-like and AI-looking UI language, including unnecessary workbench-style sections.
- Reworked account, checkout, payment verification, seller, wallet, and order screens to feel production-ready.
- Changed checkout copy to buyer-friendly language, for example: delivery, service fee, and payment total are calculated on checkout.
- Hid internal accounting values from buyer-facing checkout views, including commission, seller payout, platform revenue, provider delivery cost, and delivery markup.

3.3 Authentication and User Account Flows

- Added password visibility toggles to login and signup fields.
- Updated signup flow to support confirmPassword in line with backend changes.
- Supported email verification flow improvements after backend updates to code handling and email delivery.
- Improved account page structure into practical tabs for overview, addresses, orders, wallet, and seller setup.

3.4 Product Marketplace Experience

- Added Amazon/Jumia-style search suggestions on the landing page instead of requiring only Enter-key search.
- Added product result suggestions and related terms while users type.
- Implemented persistent product likes using backend like/unlike endpoints and login-aware error handling.
- Added sold/out-of-stock banners on product cards.
- Added low-stock messaging when one or two items remain.
- Supported discount percentage display on product cards and edit screens where the backend exposes that field.
- Added quantity controls on the product detail page and capped buyer quantity to available product stock.

3.5 Product Listing, Editing, and Seller Profile

- Fixed product creation body to match backend validation: categories, tags, numeric basePrice, numeric quantity, image arrays, gender, condition, color, and size.
- Fixed upload payload for multiple product images so files are sent as an array as required by the backend.
- Added support for multiple pictures in product listing.
- Added product status selection during creation so sellers can create listings with the desired status.
- Added seller profile listing visibility so sellers can see products they posted.
- Added listing edit route and real edit page instead of routing sellers back to a read-only product detail page.
- Added editable listing fields, including title, description, price, stock quantity, status, condition, size, color, gender, and discount percentage.

3.6 Cart and Checkout

- Added cart badge count in the header so the cart icon reflects the current number of cart items.
- Integrated cart state changes with add/remove/update flows.
- Enforced stock limits in cart and product detail quantity controls.
- Blocked checkout for sold-out items or quantities exceeding available stock.
- Supported backend rule that one checkout/order supports one seller only, with frontend seller-group checkout selection.
- Updated checkout summary to display buyer-facing values only: items total, delivery fee, service fee, and total.
- Connected checkout to POST /api/payment/orders/create and stored pending order/payment reference for verification.
- Handled backend validation errors for pickup and delivery addresses with helpful navigation to the relevant account section.

3.7 Paystack Payment Verification and Delivery Tracking

- Built payment verification page for Paystack callback/reference handling.
- Connected verification to POST /api/payment/orders/verify-payment.
- Removed confusing unavailable placeholders from payment and delivery details where values were not returned.
- Displayed real shipment/order status after successful payment.
- Connected dispatch tracking actions to shipment and sync endpoints.
- Extracted and displayed tracking URL/job ID when the backend returns Kwik tracking links.
- Ensured frontend cart state is cleared after successful order verification to reflect backend order placement behavior.

3.8 Addresses and Lagos Coordinate Requirements

- Added seller PICKUP address support with contact phone, Lagos validation, latitude, and longitude.
- Added delivery requirements for buyer SHIPPING/BILLING addresses so Kwik quote requests have phone, latitude, and longitude.
- Added 'find coordinates' and 'use current location' helpers so users do not need to manually type latitude and longitude.
- Validated that delivery addresses are within likely Lagos coordinates before saving/using them.
- Kept edit forms populated while clearing newly-created address forms after successful save.

3.9 Messaging

- Connected direct seller messaging from product detail and seller profile flows.
- Integrated backend conversation and message endpoints.
- Fixed conversation history showing generic 'User' or 'Seller' instead of the real participant name.
- Added local name persistence/merge logic for conversations so the history remains usable after returning to chats.
- Added search in the messaging section so users can find sellers/buyers across conversations.

3.10 Wallet, Banks, Withdrawals, and Seller Payouts

- Added seller wallet visibility and withdrawal history support.
- Added withdrawal to saved bank and withdrawal to a specific bank account.
- Connected supported banks endpoint to a searchable bank dropdown so users do not need to know bank codes.
- Connected POST /api/payment/banks/resolve so account name can be looked up from bank and account number.
- Displayed saved payout bank details with masked account number.
- Added guard preventing saved-bank withdrawal when no saved bank exists.
- Cleared bank and address creation forms after successful save/request.
- Updated seller sales/order summaries to show seller payout rather than gross product amount when payout fields are available.

4. Backend Work and API Flow Alignment

Backend and API work centered on making the marketplace transaction flow reliable end to end: buyer checkout, seller pickup, Kwik quote/dispatch, Paystack payment, inventory updates, shipment tracking, seller wallet settlement, and withdrawal support.

4.1 Checkout Flow

- Seller pickup address rule: sellers create a PICKUP address in Lagos with contactPhone, latitude, and longitude.
- Fallback rule: if no PICKUP address exists, backend can fall back to seller SHIPPING address where allowed.
- Buyer checkout request uses POST /api/payment/orders/create with buyer items, shipping addressId, and callbackUrl.
- Backend validation confirms buyer/session, address ownership, duplicate item merging, product existence, active status, stock availability, self-purchase blocking, and single-seller checkout.
- Backend delivery quote supports manual fee mode and live Kwik staging quote mode from seller pickup to buyer shipping address.
- Backend creates pending order with shipment status QUOTED and stores subtotal, delivery fee, total amount, pickup snapshot, and dropoff snapshot.
- Paystack amount includes item subtotal plus delivery/service fees as returned by backend.
- Frontend verifies payment, backend confirms paid amount, changes order to PROCESSING, reduces stock, records activity, and creates dispatch.
- Manual dispatch mode moves shipment to READY_FOR_PICKUP; live dispatch mode creates Kwik task and stores tracking data.

4.2 Fees, Revenue, and Settlement Logic

- Backend now calculates and stores providerDeliveryFee, deliveryMarkup, deliveryFee, serviceFee, commissionAmount, sellerPayoutAmount, platformRevenueAmount, and totalAmount.
- Frontend was updated to display only backend-returned values and not calculate fees locally.
- Buyer checkout summary was restricted to subtotal, delivery fee, service fee, and total.
- Seller-facing screens show item subtotal, commission, and seller payout.
- Seller wallet settlement uses sellerPayoutAmount after delivery rather than full item subtotal.

4.3 Auth, Messaging, Banks, and Withdrawals

- Signup backend was updated to include confirmPassword and prevent user creation on mismatch.
- Email verification setup was hardened: Redis/email delivery must succeed or newly-created users are rolled back.
- Email verification normalizes email casing and accepts codes with extra spaces/lowercase.
- Messages backend endpoints support conversations, fetching messages with a user, and sending messages.
- Withdrawal history endpoint metadata and optional DB migration support were added.
- Bank account resolve endpoint POST /api/payment/banks/resolve was added and then wired into frontend account-name lookup.

5. Product Management and Vendor Coordination Work

Beyond code delivery, PM work was required to keep the checkout/logistics/payment flow moving. This included turning backend changes into frontend requirements, identifying missing UX paths, relaying blockers, and coordinating logistics questions around Kwik.

- Clarified the new backend checkout flow and translated it into frontend tasks.
- Tracked requirements for seller pickup address, buyer shipping address, Lagos validation, phone number, latitude, and longitude.
- Coordinated fee display rules so buyer screens did not expose internal accounting fields while seller/admin screens could show payout and revenue information.
- Helped define the one-seller checkout limitation and the frontend behavior for multi-seller carts.
- Handled repeated communication/follow-ups with Kwik to unblock staging delivery quote, dispatch creation, tracking URL/job ID behavior, and logistics readiness.
- Bridged backend, frontend, and delivery-provider concerns so failures could be diagnosed from the correct side rather than treated as generic frontend errors.

6. QA, Debugging, and Production Readiness

- Ran repeated local builds and lint checks after major changes.
- Confirmed Vercel-compatible build output after replacing the Lovable config.
- Handled checkout failures caused by missing pickup/shipping coordinates and converted them into clear frontend guidance.
- Validated payment verification and delivery detail display paths after successful Paystack payment.
- Improved empty states and unavailable-data rendering so production UI does not look broken when optional backend fields are absent.
- Added README documentation explaining how to install dependencies, configure environment variables, run locally, build, preview, and deploy.

7. Deployment and Hosting Work

The frontend was migrated away from the Lovable Vite wrapper to a deployable official Vite/TanStack/Nitro setup. This makes the project easier to own, push, and publish from Git.

- Replaced @lovable.dev/vite-tanstack-config with direct Vite configuration.
- Added official plugins: tanstackStart, nitro, viteReact, Tailwind, and tsconfig paths.
- Configured Nitro preset for Vercel and pinned Vercel runtime to nodejs22.x.
- Ignored generated .vercel and .nitro build artifacts in Git and ESLint.
- Verified npm run build generates .vercel/output for public Vercel deployment.
- Documented required frontend environment variable: VITE_API_BASE_URL=https://afrab-dev-api-v1.onrender.com.

8. Detailed Deliverables Checklist

Area	Delivered Items
Authentication	Login/signup improvements, password visibility, confirmPassword compatibility, email verification flow readiness.
Products	Create, edit, categories, tags, status, multiple images, discounts, seller listing visibility, stock and sold states.
Search	Landing page suggestions, product suggestions, search navigation, marketplace filter support.
Likes	Backend like/unlike persistence, login-aware behavior, likes page support.
Cart	Header count badge, quantity controls, stock caps, clear cart behavior, grouped seller checkout.
Checkout	Backend order creation, buyer-facing fee display, one-seller checkout, Paystack redirect/reference handling.
Payments	Verify-payment page, payment summary, delivery summary, tracking status/URL display, cart state cleanup.
Addresses	PICKUP/SHIPPING/BILLING support, Lagos requirements, coordinate lookup, current location helper.
Messaging	Seller messaging, conversation history, real participant names, chat search.
Wallet	Balance display, withdrawal history, saved/specific bank withdrawal, supported banks dropdown, account resolve.
Orders/Sales	Purchase images, order status, shipment tracking, seller payout summary.
Deployment	README, Vercel-ready config, Lovable removal, build/lint verification.

9. Compensation and Scope Justification

This work should be evaluated as a cross-functional delivery effort, not as a narrow frontend task. The delivered scope included PM coordination, backend flow clarification/implementation support, API contract alignment, vendor follow-up, QA, and a complete frontend rebuild suitable for production workflows.

The frontend rebuild was outside the usual PM/backend payment expectation, but it became necessary for the product to be usable. Delivering it reduced the need for a separate frontend hire, accelerated product readiness, and made it possible to test real buyer/seller flows against the backend.

- The work converted a Lovable export into an independently maintainable local codebase.
- The work connected the frontend to real backend endpoints and current business rules.
- The work enabled production-critical flows: listing, cart, checkout, payment, delivery, wallet, withdrawal, messaging, and seller operations.
- The work included PM/vendor coordination with Kwik, which is separate from frontend implementation but essential to logistics readiness.
- The work included backend/API alignment and backend-flow updates that impacted checkout, settlement, email verification, withdrawals, and bank resolution.

10. Closing Statement

The Afra B application now has a materially stronger foundation than the original exported frontend. It can run locally, integrates with the backend, supports buyer and seller workflows, handles payment and delivery flows, includes seller wallet/withdrawal tooling, and has been prepared for public deployment.

Based on the scope documented above, the payment request covers not only PM/backend responsibilities but also the additional frontend production rebuild and integration work that was delivered to move the project toward launch readiness.